

Quelques infos pour écrire le yaml

Vérificateurs

Pour chaque colonne, on peut ajouter des vérificateurs.

- vérifier la nature d'un champ (float, integer, date) (Integer, Float, Date) et son interval de valeur (min, max)
- vérifier une expression régulière (String)
- ajouter un lien avec un référentiel (Reference)
- déclarer la récursivité d'un composant
- déclarer une hiérarchie entre composants
- vérifier la valeur en utilisant un script (le script renvoyant true) (GroovyExpression)

```
sites:
#donnée de référence avec une clef sur une colonne
OA_naturalKey:
  - zet_chemin_parent
  - zet_nom_key
OA_basicComponents:
  zet_nom_key:
    OA_mandatory
    OA_headerName: nom du site:
    OA_mandatory: true # La colonne doit être présente
    OA_required: true # Une valeur doit être fournie
  tze_type_nom:
    OA_required: true
  OA_checker:
    OA_name: OA_reference # un vérificateur de type référence
    OA_params:
      OA_reference:
        OA_name: type_de_sites # Le référentiel ciblée
        OA_isParent: true # indique que la colonne tze_type_nom contient une clef primaire
  zet_chemin_parent:
    OA_required: false
    OA_checker:
      OA_name: OA_reference # vérificateur de type référentiel
    OA_params:
      OA_reference:
        OA_name: sites # référentiel ciblé
        OA_isRecursive: true # la lien de référence est de type récursif
  date:
```

```

OA_checker:
  OA_required: true
  OA_name: OA_date
  OA_params:
    OA_pattern: dd/MM/yyyy # pattern de date attendu
    OA_min: 01/01/1980 # la valeur minimum acceptée
    OA_max: 31/12/2014 # la valeur maximum acceptée
numero:
  OA_headerName: numéro:
  OA_checker:
    OA_name: OA_integer # la valeur attendue est un entier
    OA_min: 100 # la valeur minimum acceptée

```

Colonne obligatoire

```

maColonne:
  OA_mandatory: true # default false

```

Valeur obligatoire

La notion de colonne obligatoire est à placer au niveau de la définition du composant (pas besoin de déclarer un vérificateur)

```

maColonne:
  OA_required: true # default false

```

Paramétrage des vérificateurs

On définit un vérificateur dans une section “OA_checker”. Le type de vérificateur est défini par son nom (OA_name). On peut passer des paramètres au vérificateur en renseignant la section OA_params. Les différents paramètres dépendent du type de vérificateur utilisé.

```

OA_checker:
  OA_name: OA_integer
  OA_params:

```

Lorsque l’on utilise un vérificateur, sa première fonction est de vérifier le format de la valeur en entrée : Sa seconde fonction est de transformer cette valeur , dans le cas où cela est possible,

dans une primitive acceptable dans un champ json (numeric, boolean). C'est cette valeur qui sera stockée dans le champ json en base, ou comme valeur dans les vues. Si le vérificateur est de type Reference, il existera en base de données une contrainte de type clef étrangère avec la ligne référencée.

Paramètres généraux

- OA_multiplicity :
 - MANY : La valeur est un ensemble (tableau) de valeurs. L'entrée est une chaîne ou chaque valeur est séparée par une virgule ','. Dans la base de données les valeurs de la chaîne seront enregistrées dans un tableau de valeur au format indiqué par le vérificateur. Par exemple la chaîne "2,25.3,5.8" sera traitée comme un tableau de double [2,25.3,5.8] pour un vérificateur FFloat.
 - ONE : (valeur par défaut) La valeur en entrée est considérée comme une valeur simple.
- transformations : La fonctionnalité de transformation est supprimée. Pour le moment les référentiels sont à fournir au format clef primaire.

Vérificateur de type 'Integer' et 'Float'

Ces vérificateurs servent à vérifier que les valeurs en entrée sont des nombres (respectivement des entiers ou des nombres à valeur flottante).

On peut préciser les valeurs minimum et maximum en précisant les paramètres OA_min et/ou OA_max. Les valeurs min et max doivent être du type indiqué par le vérificateur.

```
OA_checker:  
  OA_name: OA_float  
  OA_params:  
    OA_min: 12.0  
    OA_max: 25.0  
    OA_multiplicity: MANY
```

Vérificateur de chaîne ('String')

Sans vérificateur, les entrées sont traitées comme des chaînes de caractères acceptant de valeurs vides. On peut toutefois rajouter un vérificateur chaîne pour préciser des contraintes sur la chaîne. (required, multiplicity, transformation, expression régulière)

Le paramètre 'OA_pattern' permet de préciser une [expression régulière](#) qui permet de vérifier un pattern de chaîne de caractères.

```
OA_checker:
  OA_name: OA_string
  OA_params:
    OA_pattern: ^((([<>()\\[\]\\. ,;:\\s@"]+)(\\.([<>()\\[\]\\. ,;:\\s@"]+))*|(".+"))@((\\[0-
```

Ce vérificateur permet de vérifier que l'entrée est une liste d'adresse mail.

Vérificateur de date. ('Date')

Ce vérificateur permet de vérifier que la valeur en entrée est une date au format définit par le paramètre 'OA_pattern'

En base de données, dans le champs json, la date sera stockée comme une chaîne de caractères qui supporte le tri. Dans les vues, le format timestamp sera utilisé.

exemple la date 25/12/84 au format 'dd/MM/yyyy' sera stockée comme chaîne "date:1984-12-25T00:00:00:dd/MM/yyyy".

Les paramètres OA_min et OA_max permettent de spécifier l'intervale de valeur de la date. Il doivent être renseignés au même format de date.

Le paramètre OA_duration permet de définir que la valeur en entrée est une durée. Une durée est définie au sens SQL d'un [interval](#) ('1 HOUR', '2 WEEKS', '30 MINUTES').

```
OA_checker:
  OA_name: OA_date
  OA_params:
    OA_pattern: dd/MM/yyyy
    OA_min: 01/01/2004
    OA_max: 31/12/2025
    OA_duration: 1 DAY
```

Vérificateur de Boolean. ('Boolean')

Permet de vérifier que la valeur en entrée est true ou false.

Une valeur booléenne sera enregistrée dans le champ json en base de données et dans les vues.

```
OA_checker:
  OA_name: OA_boolean
```

Vérificateur de référentiel. ('Reference')

Ce vérificateur permet de vérifier que la valeur en entrée est un '[Ltree'<https://www.postgresql.org/docs/current/> Cette chaîne ne peut contenir que des termes contenant des minuscules/majuscules/chiffres/caractère_souligné(séparés par des points (.). Elle doit correspondre à la clef naturelle ou la clef hiérarchique d'une référence de type défini par le paramètre 'refType'. La section [code](#) explique comment les chaînes sont encodées pour définir des **clefs naturelles** ou des **clefs hiérarchiques**.

Le paramètre OA_reference permet de définir le référentiel contenant la ligne dont la clef naturelle est celle indiquée par la valeur en entrée.

```
zones_etudes:
  OA_required: true
  OA_checker:
    OA_name: OA_reference
    OA_params:
      OA_reference:
        OA_name: zones_etudes
```

Dans la section OA_reference on peut préciser les informations précédemment contenue dans la section composite_references OA_recursive -> le composant est récursive OA_parent -> la colonne fait référence à un référentiel parent

Il faut caster les valeurs du datum avant de les utiliser. Elles sont de type Object, et il faut les caster dans le type correspondant au checker. (par défaut String)

Vérificateur utilisant une **expression groovy**. ('GroovyExpression')

L'expression définit dans le paramètre groovy → expression; L'expression doit envoyer une valeur true/false.

```
OA_checker:
  OA_name: OA_groovyExpression
  OA_params:
    OA_groovy:
      OA_expression: >
        Set.of("", "0", "1", "2").contains(datum.SWC.get("qualité"))
```

On vérifie que la composante 'qualité' de la variable 'SWC' est vide ou "0" "1" ou "2".

Ajout de transformation à la chaîne en entrée.

- OA_groovy : permet de déclarer une transformation de la valeur avec une expression Groovy (qui doit retourner une chaîne de caractère)

La section groovy accepte trois paramètres

- OA_expression : une expression groovy (pour le checker GroovyExpression doit renvoyer true si la valeur est valide)
- OA_references : une liste de référentiels (ou data) pour lesquels on veut disposer des valeurs dans l'expression

:alert: La différence entre une section groovy de la section params d'un checker **groovy** et une section groovy de la section transformation de la section params, tient dans le fait que pour un checker groovy l'expression renvoyée est un booléen tandis que dans la transformation l'expression groovy renvoie une nouvelle valeur.

Pour les checkers GroovyExpression et les transformations Groovy, on récupère dans le script des informations :

datum : les valeurs de la ligne courante.

On récupère la valeur d'un variable-composant →

datum.get("nom de la variable").get("nom du composant")

application : le yaml de l'application

references: les valeurs d'une donnée de référence spécifique;

Il faut renseigner dans params la clef "references" qui définit

les données de références accessibles dans references.

→ references.get("nom de la reference") //référentiel déclaré dans references. return une

→ en itérant sur la liste list.collect({it.refValues.nom_de_la_colonne})

referencesValues : idem que references;

→ referencesValues.get("nom de la reference").collect({it.get("nom de la colonne")})

:alert: Les valeurs récupérées dans references et referencesValues au type indiqué par le checker. Il peut être nécessaire de les 'caster' dans ce type pour les utiliser dans des méthodes. par exemple : (String)datum.date.day + ' ' + (String)datum.date.time

:information_source: On peut aussi passer des constantes dans le script

```
OA_expression : >
import java.time.LocalDate
import java.time.format.DateTimeFormatter

LocalDate minDate = LocalDate.of(2014,1,1)
LocalDate maxDate = LocalDate.of(2022,1,1)
LocalDate date = LocalDate.parse(
    datum.date,
    DateTimeFormatter.ofPattern('dd/MM/yyyy')
)
return date.isBefore(maxDate) && date.isAfter(minDate)
```

Utilisation de validations portant sur une ou plusieurs colonnes

Les contraintes se définissent pour chacune des données de référence. Soit dans la définition de la colonne elle-même, soit dans la section validation.

Chaque règle de validation peut porter sur plusieurs colonnes de la donnée de référence. Elle comporte une description et un OA_checker (OA_eference, OA_integer, OA_float, OA_string, OA_date, OA_groovyExpression).

```
site_theme_datatype:
  OA_validations:
    projetRef: # la clef d'une validation
      OA_i18n:
        fr: "référence au projet" # la description en français
        en: "project reference" # la description en anglais
      OA_checker: # le checker de validation
        OA_name: OA_reference #Le checker à utiliser
        OA_params: # liste de paramètres (dépend du checker choisi)
          OA_reference:
            OA_name: projet #pour le checker référence la donnée référencée
      OA_columns: [nom du projet]
      # liste des colonnes sur lequel s'applique le checker
    sitesRef:
      OA_i18n:
        fr: "référence au site" # la description en français
        en: "site reference" # la description en anglais
      OA_checker:
        OA_name: OA_reference
        OA_params:
          OA_reference:
            OA_name: sites
      OA_columns: [nom du site]
```